Homework 4: Storage and Normalization

COSC 3337 — Databases — Fall 2026

Released	TBD
Due	Sunday, November 22, 2026, 11:59 pm
Total	100 points, plus 5 bonus

Before you start

The scenario. Soundwave has bought *Soundwave Live*, a small concert-ticketing startup. Soundwave Live ran its whole business out of one spreadsheet, and you have been handed its export: one MySQL table, `ticket_sale_wide`, in a new database, `soundwave_live`. Each row is one *line* of a ticket order: an order is for one concert, and can buy tickets in several seat sections.

Columns of `ticket_sale_wide`	**Meaning**
`order_id`, `order_date`	the order, and when it was placed. Order ids increase over time.
`listener_id`, `username`, `email`	the Soundwave listener who placed it
`concert_id`, `concert_date`	the concert the order is for
`artist_id`, `artist_name`	who is playing
`venue_id`, `venue_name`, `city`, `capacity`	where
`seat_section`, `price`	a seat section (`GA`, `FLOOR`, `VIP`, ...) and its price per ticket at this concert
`quantity`	how many tickets in that section this order bought

Soundwave Live's staff confirmed these **business rules**. They are the set of functional dependencies F for the whole homework:

R1	`listener_id` → `username`, `email`
R2	`artist_id` → `artist_name`
R3	`venue_id` → `venue_name`, `city`, `capacity`
R4	`concert_id` → `concert_date`, `artist_id`, `venue_id`
R5	`concert_id`, `seat_section` → `price`
R6	`order_id` → `order_date`, `listener_id`, `concert_id`
R7	`order_id`, `seat_section` → `quantity`

Part 1 is written (50 points). **Part 2** is SQL (50 points). Questions 1.4 and 1.5 analyze `ticket_sale_wide` on paper, and Part 2 then does the same work in MySQL, so do 1.5 before 2.3.

What to submit: one zip file containing exactly:

1. `hw4.pdf` — your answers to Part 1.
2. `fd_checks.sql` — your answer to Question 2.1.
3. `fix.sql` — your answer to Question 2.2.
4. `normalize.sql` — your answer to Question 2.3.
5. `lossless_check.sql` — your answer to Question 2.4.

Do not rename the four `.sql` files. The autograder looks for them by name.

Typesetting bonus (5 points).

- **By hand.** You may hand-write Part 1. Submit a PDF made with a scanning app, not a photograph, with every question clearly labeled.
- **With LaTeX (5 bonus points).** If you typeset Part 1 in LaTeX, you earn 5 bonus points. A template, `hw4_template.tex`, is in the `HW4` folder. Submit the compiled PDF, not the `.tex` file.

Do Part 0 first, and early, so we have time to fix any setup problems.

Part 0 — Setup *(0 points, required)*

0.1 Get the starter files

On your VM, go to your copy of the course repository, pull the new homework from `upstream`, and load the new database:

```
cd COSC3337-FA26-homeworks
git pull upstream main
cd HW4
bash setup_db.sh
```

`setup_db.sh` loads `soundwave_live`, installs the Python packages the tests need, and checks that everything connects. It is safe to run again at any time and restores `ticket_sale_wide` to the original export, so it is your repair command. It also *deletes* any tables you created in `soundwave_live`, but never your `.sql` files.

0.2 Run the tests

```
python3 -m pytest tests/ -v
```

Every test should run and *fail* with a message about your answer (the starter files contain placeholders). That means your setup works. Any other error (cannot connect, access denied, module not found) is a setup problem. Re-run `setup_db.sh`, and if that does not fix it, email me or come to office hours.

0.3 Back up your work

Commit and push to your private `cosc3337-work` repository every time you stop working. From inside `HW4`:

```
git add *.sql
git commit -m "<what you did this session>"
git push origin main
```

Your VM has no backup.

Part 1 — Written *(50 points)*

Show your work for every calculation.

1.1 RAID *(11 points)*

Soundwave Live is buying a server with four 4 TB disks.

(a) *(3 pts)* For RAID 0 (block striping across all four disks), RAID 1 (two mirrored pairs, with blocks striped across the pairs) and RAID 5 (block-interleaved distributed parity), give the usable capacity and the number of disk failures the layout survives *no matter which* disks fail.

(b) *(4 pts)* Under RAID 5, one stripe holds three data blocks on disks 0, 1 and 2 and their parity block on disk 3. With blocks shortened to 8 bits:

$$D_0 = \texttt{10110010} \qquad D_1 = \texttt{01101100} \qquad D_2 = \texttt{11100001}$$

1. Compute the parity block $P = D_0 \oplus D_1 \oplus D_2$, showing each step.
2. Disk 1 fails. Show how the system rebuilds D_1 from the blocks that survive, and check that you got D_1 back.

(c) *(4 pts)* Soundwave Live will store two things on this server: the *play log*, which receives a small write every time anyone plays a song, and the *ticket archive*, which is written once a night and read by reports all day. Choose RAID 1 or RAID 5 for each, and justify both choices.

1.2 Slotted pages *(7 points)*

`ticket_sale_wide` has variable-length records (`email` and `venue_name` vary in length), so a DBMS would store them in slotted pages. A small 240-byte block holds four of its records in a slotted page. The header takes 4 bytes plus 4 bytes per slot, and records are packed at the end of the block. Right now the end of free space is 130, leaving 110 free bytes.

Slot	(offset, size) now	After (a)	After (b)
1	(215, 25)		
2	(175, 40)		
3	(160, 15)		
4	(130, 30)		
5	—		
End of free space	130		
Free bytes	110		

(a) *(4 pts)* Delete the record in slot 1 and compact the page. Fill in the header.

(b) *(3 pts)* Insert a 50-byte record into a new slot 5. Fill in the header.

1.3 File organization *(7 points)*

For each workload, choose a *heap* file, a *sequential* file or *columnar* storage, and justify your choice.

(a) *(2 pts)* Soundwave's play log: millions of rows appended every day, read only by a nightly job that scans all of it.

(b) *(3 pts)* Ticket orders, stored by `order_id`. Invoicing reads them in ranges ("orders 101000 to 101999"), and new orders almost always arrive in increasing `order_id`. Also say what happens when a late order with a *smaller* `order_id` arrives and the block it belongs in is full.

(c) *(2 pts)* An analytics query computes the average `price` for each `seat_section` over 500 million rows of a 16-column table like `ticket_sale_wide`. If all columns are the same width, roughly what fraction of a row layout's bytes does your choice read?

1.4 Functional dependencies in the export *(9 points)*

Here are four rows of `ticket_sale_wide` (some columns omitted):

order_id	listener_id	email	concert_id	seat_section	price	quantity
100856	L0071	silvermaple72_9@example.com	C002	FLOOR	89.50	1
100856	L0071	silvermaple72_9@example.com	C002	UPPER	63.00	1
101276	L0071	silvermaple72@example.com	C021	UPPER	46.50	2
101386	L0071	silvermaple72@example.com	C046	LOWER	60.50	4

(a) *(2 pts)* These rows already show the damage done by one kind of anomaly. Which kind, and how did it most likely happen?

(b) *(4 pts)* Using F, compute $\{\texttt{order_id}\}^+$ and $\{\texttt{order_id}, \texttt{seat_section}\}^+$ with the attribute closure algorithm. Show which rule you apply at each step. What do the results tell you about each attribute set?

(c) *(3 pts)* Show that $\{\texttt{order_id}, \texttt{seat_section}\}$ is the *only* candidate key of `ticket_sale_wide`.

1.5 BCNF decomposition *(16 points)*

(a) *(3 pts)* Which of R1–R7 violate BCNF on `ticket_sale_wide`? Justify with your answers to 1.4.

(b) *(10 pts)* Decompose `ticket_sale_wide` into BCNF with the BCNF decomposition algorithm. For each step, give the relation you split, the violating dependency $\alpha \to \beta$, and the two relations it produces, $(\alpha \cup \beta)$ and $(R - (\beta - \alpha))$. Finish with the list of final relations, with each one's key underlined.

At each step, split on a rule from R1–R7 that lies entirely inside the relation and violates BCNF there, choosing one whose right side contains no attribute from the left side of another rule still inside the relation. A relation is finished when none of R1–R7 violates BCNF in it.

(c) *(3 pts)* Choose one step of your decomposition and show, with the test from lecture, that it is lossless.

Part 2 — SQL *(50 points)*

The files

File	What it is	You...
`fd_checks.sql`	2.1: five queries	**edit and submit**
`fix.sql`	2.2: statements that repair the export	**edit and submit**
`normalize.sql`	2.3: statements that build your BCNF tables	**edit and submit**
`lossless_check.sql`	2.4: one query	**edit and submit**
`soundwave_live.sql`	the export, loaded by `setup_db.sh`	don't edit
`tests/`	the tests	run

Run a script against the database with, for example,

```
mysql soundwave_live < fix.sql
```

Do not write `USE` in any file, and do not create other databases: the autograder runs your files against its own copy of `soundwave_live`.

How grading works

The autograder runs the same tests as `tests/`, on a **different copy of the data**: different listeners, concerts, orders, and different broken rows. Answers copied from your copy of the data will fail. Each line of the tables below is graded on its own, except where a line says otherwise. Each of 2.1–2.3 starts from freshly loaded data, so a mistake in one of them does not cost you points in another. 2.4 uses the tables your `normalize.sql` builds.
The tests reload `soundwave_live` as they run. When they finish, it is restored to the original export, so any tables you created are gone. Re-run your scripts if you want them back.

2.1 Find the rule violations *(15 points)*

A spreadsheet cannot enforce a functional dependency, so before trusting the export, you check it. In `fd_checks.sql`, write one query under each header. Each query finds the *violations* of one rule: the left-hand-side values that appear in `ticket_sale_wide` with two or more different combinations of right-hand-side values.

Query	Checks	Returns exactly these columns, in this order and sorted by them
Q1	R1	`listener_id`
Q2	R3	`venue_id`
Q3	R4	`concert_id`
Q4	R5	`concert_id`, `seat_section`
Q5	R6	`order_id`

Each violating value appears once. A value that always appears with the same right-hand side does not appear at all. Keep the `-- Q1` ... `-- Q5` header lines exactly as they are: the autograder splits the file on them.

The tests run your queries on your copy of the export and on a second copy in which all five rules are broken.

What is tested	Points
Each of Q1–Q5 returns exactly the right rows, on both copies	5×3

2.2 Repair the export *(10 points)*

Write `fix.sql`: one or more `UPDATE` statements that repair `ticket_sale_wide` in place, so that every business rule holds. Use your queries from 2.1 to find out what needs repairing.

When a left-hand-side value appears with more than one right-hand side, the correct right-hand side is the newest one: the one on that value's row with the largest `order_id`. Change nothing else. The autograder's copy is broken in the same rules as yours, but in different rows, so your statements must work for *any* value, not just the ones you found.

A MySQL restriction

MySQL refuses an `UPDATE` whose `WHERE` or `SET` clause has a subquery that reads the table being updated (error 1093, *"You can't specify target table ... for update in FROM clause"*). A subquery joined to the table in the `UPDATE` itself is allowed.

Re-run `setup_db.sh` (or `mysql < soundwave_live.sql`) before each attempt, so you always start from the original export. Afterwards, every query in `fd_checks.sql` should return no rows.

What is tested	Points
`fix.sql` runs, and afterwards every rule R1–R7 holds	4
Every repaired value is the newest one, and nothing else changed: the table matches the correctly repaired export exactly	4
No rows were added or deleted (*only awarded if every rule holds*)	2

2.3 Normalize *(20 points)*

Write `normalize.sql`, which turns your answer to 1.5 into real tables: it creates your BCNF relations in `soundwave_live` and fills them from the repaired `ticket_sale_wide`. The autograder runs it on a *correctly* repaired export, so 2.3 does not depend on your answer to 2.2.

Rules for normalize.sql (checked by the autograder)

- Name your tables anything except `ticket_sale_wide`. Every base table in `soundwave_live` other than `ticket_sale_wide` counts as part of your design, so do not leave scratch tables behind.
- Use *only* the 16 column names of `ticket_sale_wide`, with the same types: no new columns (no surrogate ids), no renaming. Every one of the 16 must appear in at least one table.
- Every table has a `PRIMARY KEY`.
- When a table contains all the columns of another table's primary key, and those columns are not exactly its own primary key, declare a `FOREIGN KEY` on those columns referencing

the other table.
- Fill every table from `ticket_sale_wide` with `INSERT ... SELECT`. Do not change or drop `ticket_sale_wide`.
- Start with `DROP TABLE IF EXISTS` for your tables, so the file can run again and again. Drop child tables before the tables they reference.

The checks use F (R1–R7). In (b), a table *stores rule Ri redundantly* if it contains Ri's left side and at least one attribute of its right side, but Ri's left side is not a superkey of the table.

What is tested	Points
(a) `normalize.sql` runs; the tables use exactly the 16 columns; `ticket_sale_wide` is unchanged	2
(b) For each of R1–R6, no table stores it redundantly	6×1
(c)–(f) are only awarded if at least 4 of the 6 checks in (b) pass.	
(c) Every table is in BCNF	2
(d) Joining all your tables gives back exactly the rows of `ticket_sale_wide`	6
(e) Every table's primary key is a candidate key of that table	2
(f) Every foreign key required by the rules above is declared	2

2.4 Prove it is lossless *(5 points)*

In 1.5 you argued that one step of your decomposition is lossless. Now check the whole decomposition against the data. Write `lossless_check.sql`: one query (it may start with `WITH`) that returns

- every row of `ticket_sale_wide` that joining your tables does not produce, and
- every row that joining your tables produces that is not in `ticket_sale_wide`.

When your decomposition is lossless, it returns no rows. The columns it returns are up to you. The MySQL on your VM supports `EXCEPT`.

To test it, the autograder builds your tables with *your* `normalize.sql` and then damages them.

What is tested	Points
Returns at least one row after one row is deleted from your largest table	2
Returns at least one row after one value outside a primary key is changed	2
Returns no rows for your tables as `normalize.sql` built them (*only awarded if the first test passes*)	1

Before you submit

1. `python3 -m pytest tests/ -v` passes.
2. Starting from a fresh `setup_db.sh`, your files run without errors in this order: `fix.sql`, `normalize.sql`, `lossless_check.sql`.
3. Your zip contains exactly `hw4.pdf` and the four `.sql` files, with their original names.